

文章目录

一、绪论

1、操作系统的定义

操作系统是将系统中的各种软、硬资源有机地组合成一个整体，合理地组织计算机的工作流程，为用户提供方便、快捷、友好的应用程序使用接口。

2、操作系统的特征

- 并发性
- 共享性
- 不确定性

3、操作系统的功能

- 管理处理机
- 管理存储器
- 管理设备
- 管理信息(或文件)

4、操作系统的分类

- (1)批量操作系统
- (2)分时操作系统
- (3)实时操作系统
- (4)个人计算机操作系统
- (5)网络操作系统
- (6)分布式操作系统

二、处理机的态及中断

处理机的态

- 管态：又称系统态，是操作系统的管理程序执行时机器所处的状态。在此状态下，中央处理机可以使用全部机器指令，包括一组特权指令，可以使用所有的资源，允许访问整个存储区。
- 用户态：又称目态，是用户程序执行时机器所处的状态。在此状态下，禁止使用特权指令，不能直接取用资源与改变机器状态，并且只允许用户程序访问自己的存储区域。

特权指令

在核态下操作系统可以使用所有指令，包括一组特权指令。范围：

- 1.改变机器状态的指令；
- 2.修改特殊寄存器的指令；
- 3.涉及外部设备的I/O指令；

中断概念

中断，是指某个事件(例如电源断电、定点加法溢出或I/O设备传输结束等)发生时，系统中止现行程序的运行、引出处理该事件的程序进行处理，处理完毕后返回断点，继续执行。

三、接口

1、用户接口

用户接口是操作系统提供给用户与计算机打交道的外部机制。用户能够通过用户接口和系统提供的手段来控制用户所在的系统。操作系统的用户接口分为以下两类。

- 操作接口：用户通过这个操作接口来组织自己的工作流程和控制程序的运行。
- 程序接口：任何一个用户程序在其运行过程中，可以使用操作系统提供的功能调用来请求操作系统的服务(如申请主存、使用各种外设、创建进程或线程等)不论哪类操作系统都必须同时提供操作接口和程序接口。

2、系统调用

系统调用提供了用户程序与操作系统之间的接口。

实现过程：在用户程序中，在请求操作系统服务的地方安排一条系统调用，当程序执行到这一点命令时发生中断，系统由用户态转为管态，操作系统按照系统调用的功能号去执行程序，完成了用户所需服务功能之后，返回到用户程序的断点继续执行。

四、进程的管理

1、进程的定义

1. 进程是一个程序与其数据一道通过处理机的执行所发生的活动。
2. 进程是一个程序在给定活动空间和初始环境下，在一个处理机上的执行过程。

2、进程的组成

进程（进程实体）由程序段、数据段和PCB(进程控制块)三部分组成。

3、进程的状态及转换

在一个进程的活动期间至少具备3种基本状态，即就绪状态、运行状态、等待状态(又称阻塞状态)。

- 就绪状态(ready)
当进程获得了除CPU之外所有的资源，它已经准备就绪，一旦得到了CPU控制权，就可以立即运行，该进程所处的状态为就绪状态。
- 运行状态(running)
进程通过进程调度和处理机分派后，得到中央处理机的控制权，该进程对应的程序正在处理机运行，它所处的状态称为运行状态。
- 等待状态(wait)
若以进程正在等待某一事件(如等待输入/输出操作的完成)而暂时停止执行。这时，即使给它CPU控制权，它也无法执行，则称该进程处于等待状态，又可称为阻塞状态。

进程的状态随着自身的推进和外界条件的变化而发生变化，如图所示：



4、进程控制块

进程控制块（Process Control Block,PCB）是一个数据结构，是标识进程存在的实体。系统创建以一个进程时，必须为它设置一个PCB，然后根据PCB的信息对进程实施控制和管理。进程任务完成时，系统撤销它的PCB，进程也随之消亡。

5、进程控制

进程控制一般由OS内核中的原语实现。

用于进程控制的原语：创建原语、撤销原语、阻塞原语、唤醒原语等。

- 1、 创建原语：创建一个就绪状态的进程，使进程从创建状态变迁为就绪状态。
- 2、 撤销原语：使进程从执行状态变迁为完成状态。
- 3、 阻塞原语：使进程从运行状态变迁为阻塞状态。
- 4、 唤醒原语：使进程从阻塞状态变迁为就绪状态。

6、进程互斥与同步

- 互斥：若干个进程都要使用某一共享资源时，任何时刻最多只允许一个进程去使用该资源，其他要使用它的进程必须等待，直到该资源的占用者释放了该资源。
- 同步：一个进程依赖另一个进程的消息，当一个进程没有得到另一个进程的消息时应等待，直到消息到达才被唤醒。

7、临界资源和临界区

- 临界资源：一次仅允许一个进程使用的资源。
- 临界区：一组进程共享某一临界资源，这组进程的每个进程都包含的一个访问临界资源的程序段（并发进程中与共享变量有关的程序段）

8、PV操作

PV操作又称wait,signal原语。

主要是操作进程中对进程控制的信息量的加减控制。

- wait用法：
wait(num),num是目标参数，wait的作用是使其（信息量）减一。
如果信息量 ≥ 0 ，则该进程继续执行；否则该进程置为等待状态，排入等待队列。
- signal用法：
signal(num),num是目标参数，signal的作用是使其（信息量）加一。
如果信息量 > 0 ，则该进程继续执行；否则释放队列中第一个等待信号量的进程。

9、线程与进程的区别

线程：有时被称为轻量级进程(Lightweight Process, LWP)，是程序执行流的最小单元。线程是进程中的一个实体，是被系统独立调度和分派的基本单位，线程自己不拥有系统资源，只拥有一点儿在运行中必不可少的资源，但它可与同属一个进程的其它线程共享进程所拥有的全部资源。

区别：

- （1）进程是资源的分配和调度的一个独立单元；而线程是CPU调度的基本单元。
- （2）进程间相互独立进程，进程之间不能共享资源，一个进程至少有一个线程，同一进程的各线程共享整个进程的资源。
- （3）线程是轻量级的进程，创建和销毁所需要的时间比进程小很多。

五、处理机的调度

1、作业调度

对存放在外存的大量的作业，以一定的策略进行挑选，分配主存等必要资源，建立作业对应的进程，使其投入运行。

2、进程调度

进程调度程序按一定的策略，动态地把处理机分配给处于就绪队列中的某一个进程，以使之执行。

3、中级调度

为了提高内存利用率和系统吞吐量，将暂不能运行的进程调至外存等待（挂起），当运行条件具备后，将其再次调入内存，等待进程调度（解挂）。

4、进程调度算法

操作系统的常见进程调度算法：

1.先来先服务

先来先服务（FCFS，first come first served）调度算法是按作业来到的先后次序进行调度的。

算法原理：进程按照它们请求CPU的顺序使用CPU.就像你买东西去排队，谁第一个排，谁就先被执行，在它执行的过程中，不会中断它。当其他人也想进入内存被执行，就要排队等着，如果在执行过程中出现一些事，他现在不想排队了，下一个排队的就补上。此时如果他又想排队了，只能站到队尾去。

算法优点：易于理解且实现简单，只需要一个队列（FIFO），且相当公平

算法缺点：比较有利于长进程，而不利于短进程，有利于CPU 繁忙的进程，而不利于I/O 繁忙的进程

2.最短作业优先

短作业优先（SJF，Shortest Job First）是对FCFS算法的改进，其目标是减少平均周转时间。

算法原理：对预计执行时间短的进程优先分派处理机。通常后来的短进程不抢先正在执行的进程。

算法优点：相比FCFS 算法，该算法可改善平均周转时间和平均带权周转时间，缩短进程的等待时间，提高系统的吞吐量。

算法缺点：对长进程非常不利，可能长时间得不到执行，且未能依据进程的紧迫程度来划分执行的优先级，以及难以准确估计进程的执行时间，从而影响调度性能。

3.最高响应比优先法

最高响应比优先法（HRRN，Highest Response Ratio Next）是对FCFS方式和SJF方式的一种综合平衡。FCFS方式只考虑每个作业的等待时间而未考虑执行时间的长短，而SJF方式只考虑执行时间而未考虑等待时间的长短。因此，这两种调度算法在某些极端情况下会带来某些不便。HRRN调度策略同时考虑每个作业的等待时间长短和估计需要的执行时间长短，从中选出响应比最高的作业投入执行。这样，即使是长作业，随着它等待时间的增加， W / T 也就随着增加，也就有机会获得调度执行。这种算法是介于FCFS和SJF之间的一种折中算法。

算法原理：响应比R定义如下： $R = (W+T) / T = 1+W/T$

其中T为该作业估计需要的执行时间，W为作业在后备状态队列中的等待时间。每当要进行作业调度时，系统计算每个作业的响应比，选择其中R最大者投入执行。

算法优点：由于长作业也有机会投入运行，在同一时间内处理的作业数显然要少于SJF法，从而采用HRRN方式时其吞吐量将小于采用SJF法时的吞吐量。

算法缺点：由于每次调度前要计算响应比，系统开销也要相应增加。

六、死锁

死锁：是指多个进程因竞争资源而造成的一种僵局（相互等待），若无外力作用，这些进程都将无法前往执行。

1、死锁产生的原因和条件

- 1)、系统资源不足。
- 2)、进程推进顺序不合理。

产生死锁的4个必要条件：

- (1) 互斥条件：一个资源每次只能被一个进程使用。
- (2) 不剥夺条件：进程已获得的资源，在未使用完之前，不能被强行剥夺，只能在进程使用完时由自己释放。
- (3) 占有并等待条件：一个进程因请求资源而阻塞时，对已获得的资源保持不放。
- (4) 环路条件：系统中若干进程组成环路，该环路中每个进程都在等待下一个进程所占有的资源。

2、死锁的避免

(1) 破坏“互斥”条件:就是在系统里取消互斥。若资源不被一个进程独占使用，那么死锁是肯定不会发生的。但一般“互斥”条件是无法破坏的。因此，在死锁预防里主要是破坏其他三个必要条件，而不去涉及破坏“互斥”条件。

(2) 破坏“请求和保持”条件:在系统中不允许进程在已获得某种资源的情况下，申请其他资源。即要想出一个办法，阻止进程在持有资源的同时申请其他资源。

方法一：所有进程在运行之前，必须一次性地申请在整个运行过程中所需的全部资源。这样，该进程在整个运行期间，便不会再提出资源请求，从而破坏了“请求”条件。系统在分配资源时，只要有一种资源不能满足进程的要求，即使其它所需的各资源都空闲也不分配给该进程，而让该进程等待。由于该进程在等待期间未占有任何资源，于是破坏了“保持”条件。

该方法优点：简单、易行且安全。

缺点：a.资源被严重浪费，严重恶化了资源的利用率。b.使进程经常会发生饥饿现象。

方法二：要求每个进程提出新的资源申请前，释放它所占有的资源。这样，一个进程在需要资源S时，须先把它先前占有的资源R释放掉，然后才能提出对S的申请，即使它可能很快又要用到资源R。

(3) 破坏“不可抢占”条件：允许对资源实行抢夺。

方法一：如果占有某些资源的一个进程进行进一步资源请求被拒绝，则该进程必须释放它最初占有的资源，如果有必要，可再次请求这些资源和另外的资源。

方法二：如果一个进程请求当前被另一个进程占有的一个资源，则操作系统可以抢占另一个进程，要求它释放资源。只有在任意两个进程的优先级都不相同的条件下，该方法才能预防死锁。

(4) 破坏“循环等待”条件：将系统中的所有资源统一编号，进程可在任何时刻提出资源申请，但所有申请必须按照资源的编号顺序（升序）提出。这样做就能保证系统不出现死锁。

七、主存管理

1、分区算法

1.首次适应算法

首次适应算法（first fit, FF）：要求，空闲分区链以地址递增的顺序链接。每次从链首开始，直到找到第一个能满足要求的空闲分区为止。简单来说，就是，每次都从第一个开始顺序查找，找到一块区域可以满足要求的。

优点：优先利用内存中低址部分的空闲分区，从而保留了高址部分的大空闲区，这为以后到达的大作业分配大的内存空间创造了条件。

缺点：低址部分不断被划分，会留下许多难以利用的，很小的空闲分区，称为碎片。而每次查找又都是从低址部分开始的，这无疑又会增加查找可用空闲分区时的开销。

2.最佳适应算法

最佳适应算法（best, BF）：将所有空闲分区按照空闲分区容量大小从小到大的顺序连接起来，形成一个空闲分区链。即，每次都是找空间容量不但可以满足要求的空闲区，而且该空闲分区的容量还要最接近要求的容量大小。

- 优点：每次分配给文件的都是最合适该文件大小的分区。
- 缺点：内存中留下许多难以利用的小的空闲区（外碎片）。

3.最坏适应算法

最坏适应算法（worst，WF）与BF算法相反，WF算法是按照空闲分区容量从大到小的顺序连接起来的，而且每次找空闲分区的时候也是按照空闲分区容量最大的。

- 特点：尽可能的分配大的分区。
- 缺点：使得内存缺乏大分区，可能使得后续到来的大作业无法装入内存。

2、段式和页式的区别

1. 页式和段式管理都提供了内外存统一管理的虚存实现，但分页是出于系统管理的需要，分段是出于用户应用的需要：
2. 一条指令或一个操作数可能会跨越两个页的分界处，而不会跨越两个段的分界处。
3. 页式虚存只交换固定大小的页，需要多次缺页中断才能把所需信息完整地调入内存，而段式虚存每次交换得到是一段有意义的信息。
4. 无法通过页面共享具有完整逻辑功能的子程序或数据块，而段则可以。
5. 页大小是系统固定的，而段大小则通常不固定，对需要不断增加或吸收新数据的段十分有利。
6. 分页是一维的，各个模块在链接时必须组织成同一个地址空间；分段是二维的，各个模块在链接时可以每个段组织成一个地址空间，反映了程序的逻辑结构。
7. 通常段比页大，因而段表比页表短，可以缩短查找时间，提高访问速度。

3、页面置换算法

1.先进先出置换算法

先进先出置换算法（FIFO）：是最简单的页面置换算法。这种算法的基本思想是：当需要淘汰一个页面时，总是选择驻留主存时间最长的页面进行淘汰，即先进入主存的页面先淘汰。其理由是：最早调入主存的页面不再被使用的可能性最大。即优先淘汰最早进入内存的页面。（往前看）

访问页面	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
物理块1	7	7	7	2		2	2	4	4	4	0			0	0			7	7	7
物理块2		0	0	0		3	3	3	2	2	2			1	1			1	0	0
物理块3			1	1		1	0	0	0	3	3			3	2			2	2	1
缺页否	√	√	√	√		√	√	√	√	√	√			√	√			√	√	√

2.最近最久未使用算法

最近最久未使用（LRU）算法：这种算法的基本思想是：利用局部性原理，根据一个作业在执行过程中过去的页面访问历史来推测未来的行为。它认为过去一段时间里不曾被访问过的页面，在最近的将来可能也不会再被访问。所以，这种算法的实质是：当需要淘汰一个页面时，总是选择在最近一段时间内最久不用的页面予以淘汰。即淘汰最近最长时间未访问过的页面。（往前看）

访问页面	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
物理块1	7	7	7	2		2		4	4	4	0			1		1		1		
物理块2		0	0	0		0		0	0	3	3			3		0		0		
物理块3			1	1		3		3	2	2	2			2		2		7		
缺页否	√	√	√	√		√		√	√	√	√			√		√		√		

3.最佳置换算法

最佳置换算法（OPT）（理想置换算法）：从主存中移出永远不再需要的页面；如无这样的页面存在，则选择最长时间不需要访问的页面。于所选择的被淘汰页面将是以后永不使用的，或者是在最长时间内不再被访问的页面，这样可以保证获得最低的缺页率。即被淘汰页面是以后永不使用或最长时间内不再访问的页面。（往后看）

访问页面	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
物理块1	7	7	7	2		2		2			2			2				7		
物理块2		0	0	0		0		4			0			0				0		
物理块3			1	1		3		3			3			1				1		
缺页否	√		√	√		√		√			√			√				√		

八、设备管理

1、I/O设备类型

I/O设备包括输入设备和输出设备。

- 输入设备：将外部世界的信息输入到计算机，如键盘、数字化仪等；
- 输出设备：将计算机加工好的信息输出给外部世界，如打印机、数模转换器等；

2、I/O设备控制方式

1.轮询方式的 I/O 操作

对I/O设备的程序轮询的方式，是早期的计算机系统对I/O设备的一种管理方式。它定时对各种设备轮流询问一遍有无处理要求。轮流询问之后，有要求的，则加以处理。在处理I/O设备的要求之后，处理机返回继续工作。尽管轮询需要时间，但轮询要比I/O设备的速度要快得多，所以一般不会发生不能及时处理的问题，I/O操作的时效性是可以保证的。但是处理器的速度再快，能处理的输入输出设备的数量也是有一定限度的。而且，程序轮询会占据CPU相当一部分处理时间，因此程序轮询是一种效率较低的方式，在现代计算机系统中已很少应用。

2.中断方式的 I/O 操作

处理器与 I/O 设备间几个数量级的速度差异是 I/O 操作中存在的重要矛盾，是设备管理要解决的一个重要问题。为了提高整体效率，减少在程序直接控制 I/O 设备与处理器进行数据交互是很必要的。在I/O设备中断方式下，中央处理器与I/O设备之间数据的传输步骤如下：

- (1)在某个进程需要数据时，发出指令启动输入输出设备准备数据
- (2)在进程发出指令启动设备之后，该进程放弃处理器，并由操作系统将进程置为阻塞状态，等待相关I/O操作完成。此时，进程调度程序会调度其他就绪进程使用处理器。
- (3)当I/O操作完成时，输入输出设备控制器通过中断请求线向处理器发出中断信号，处理器收到中断信号之后，转向预先设计好的中断处理程序，对数据传送工作进行相应的处理。
- (4)数据准备完成后，OS将阻塞的进程唤醒，将其转入就绪状态。在随后的某个时刻，进程调度程序会选中该进程继续工作。

3.DMA 方式的I/O 操作

直接内存存取技术是指，数据在内存与I/O设备间直接进行成块传输。该技术基于 DMA 设备，将 CPU 从简单的数据传输工作中解放了出来。

DMA有两个技术特征，首先是直接传送，其次是块传送。所谓直接传送，即在内存与IO设备间传送一个数据块的过程中，不需要CPU的任何中间干涉，只需要CPU在过程开始时向设备发出“传送块数据”的命令，然后通过中断来得知过程是否结束和下次操作是否准备就绪，当然这里的中断是 DMA 设备向 CPU 发出的而不是设备控制器。

九、文件系统

- 文件：文件是信息的一种组织形式，是存储在外存上的具有标识名的一组相关信息集合；
- 文件系统：操作系统中负责管理和存取文件信息的软件机构；

1、文件的结构

- 逻辑结构：指一个文件在用户面前所呈现的形式。

逻辑结构有两种形式：①记录式文件(有结构式文件).②字符流式文件（无结构式文件）,也称流式文件。

- 物理结构：指文件在文件存储器上的存储形。所谓文件系统的物理结构是指数据存放在硬盘上时硬盘磁粉的排列形状。

物理结构的形式：①连续文件结构②串联文件结构③索引文件结构④散列文件结构。